

IKE – Internet Key Exchange

IKE – Internet Key Exchange

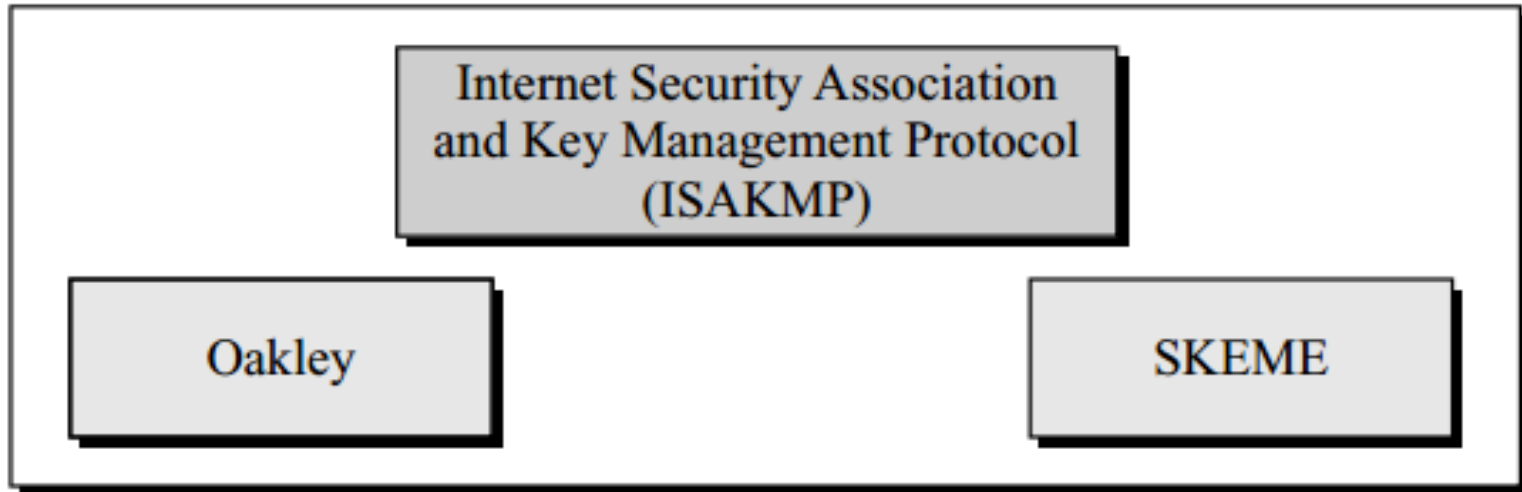
- Security associations are by IP address
- What if your address changes?
 - Traveler with laptop wants to join a company's VPN
- IKE can authenticate endpoints and automatically setup security associations
 - Can use public key infrastructure (X.509) to authenticate endpoint identity
 - Can also use pre-shared private keys

Internet Key Exchange (IKE)

- Does authentication and negotiates private keys
- Exchange and negotiate security policies
- Establish security sessions
 - Identified as *Security Associations*
- Key exchange
- Key management
- Can be used outside IPsec as well

IKE components

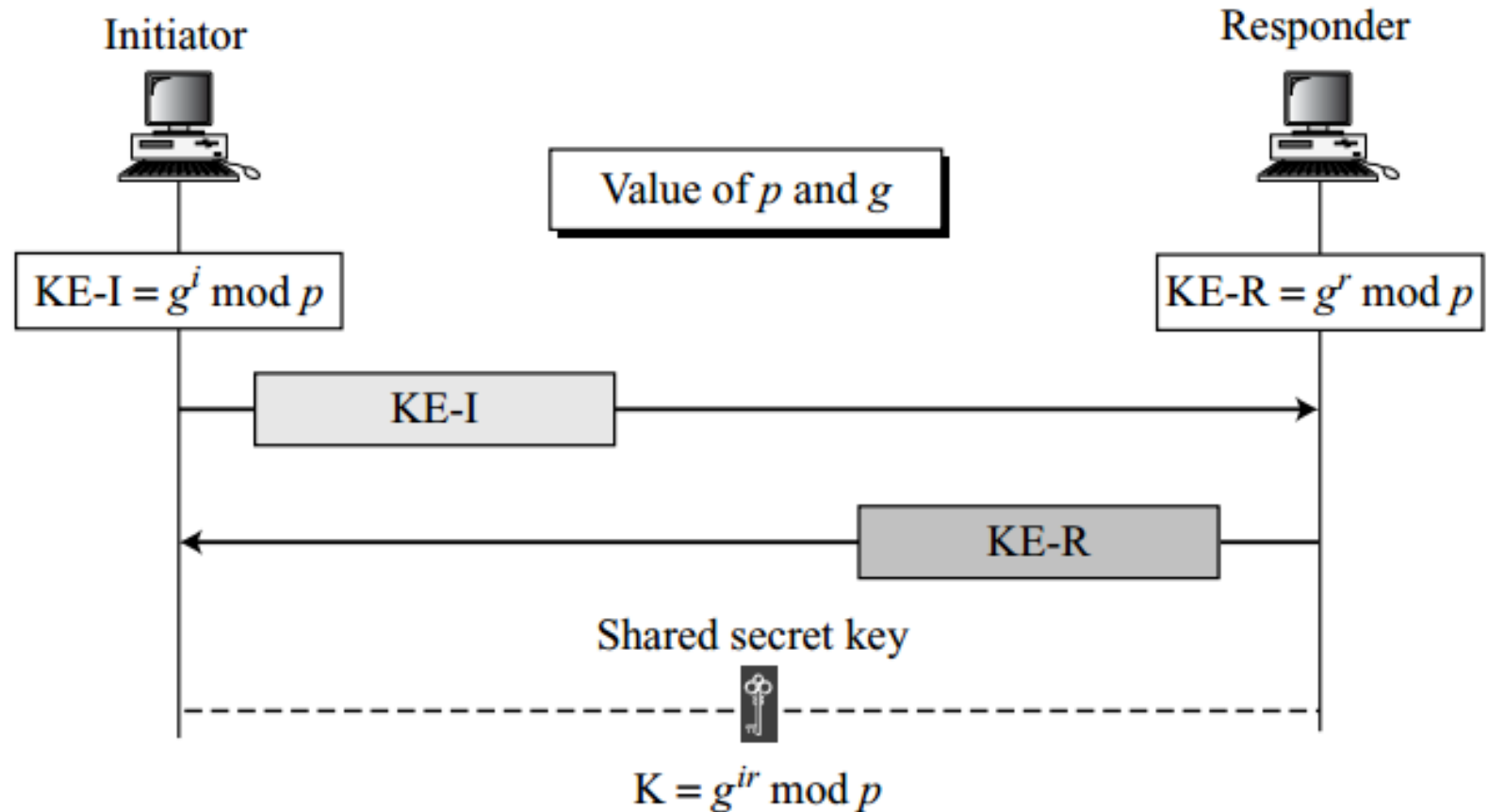
Internet Key Exchange (IKE)



Oakley protocol is a key creation protocol based on **the improved Diffie-Hellman key-exchange method**

SKEME uses public-key encryption for entity authentication in a key-exchange protocol.

Improved Diffie-Hellman Key Exchange

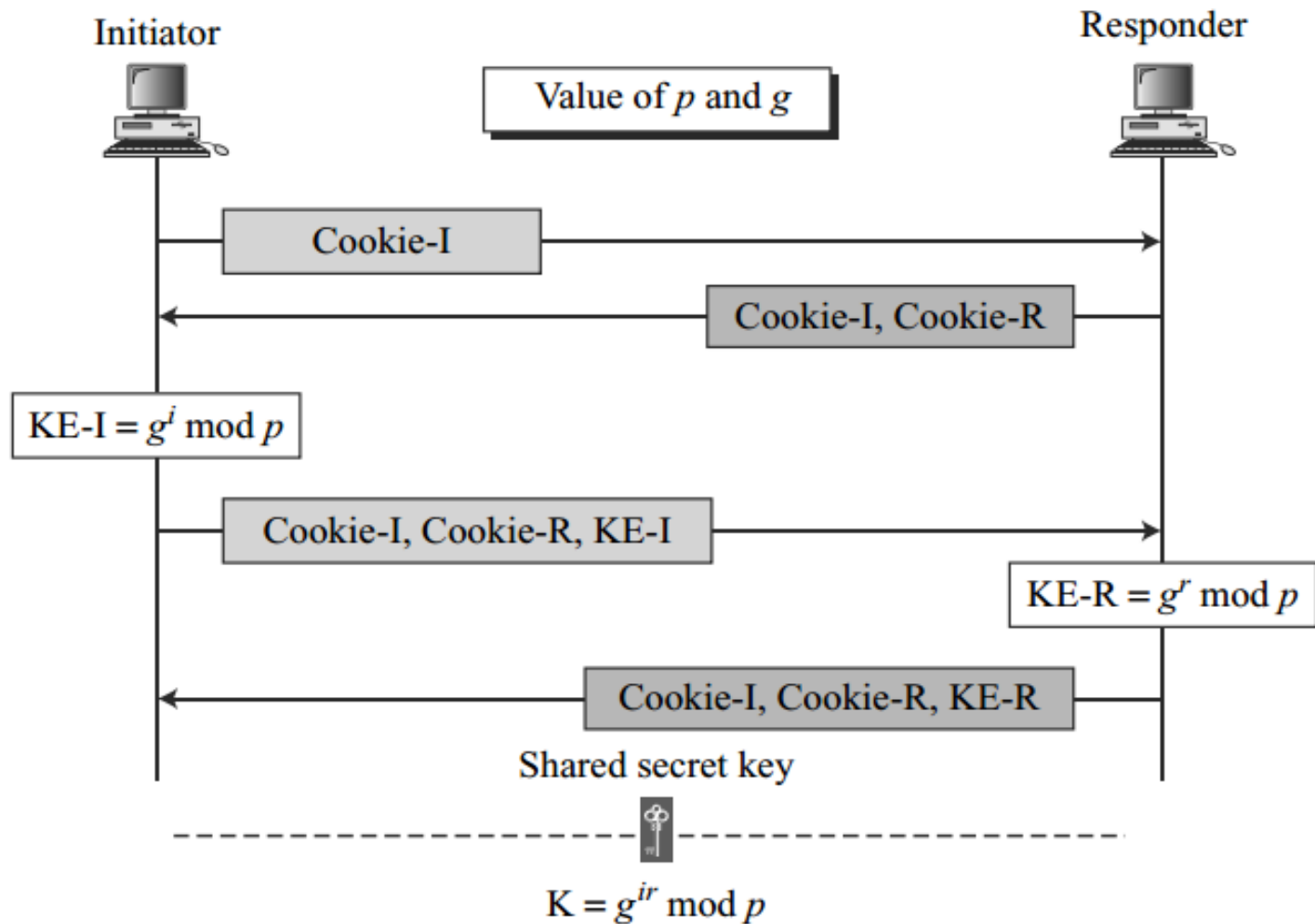


It is susceptible to [clogging attack](#) or [denial-of service](#) attack.

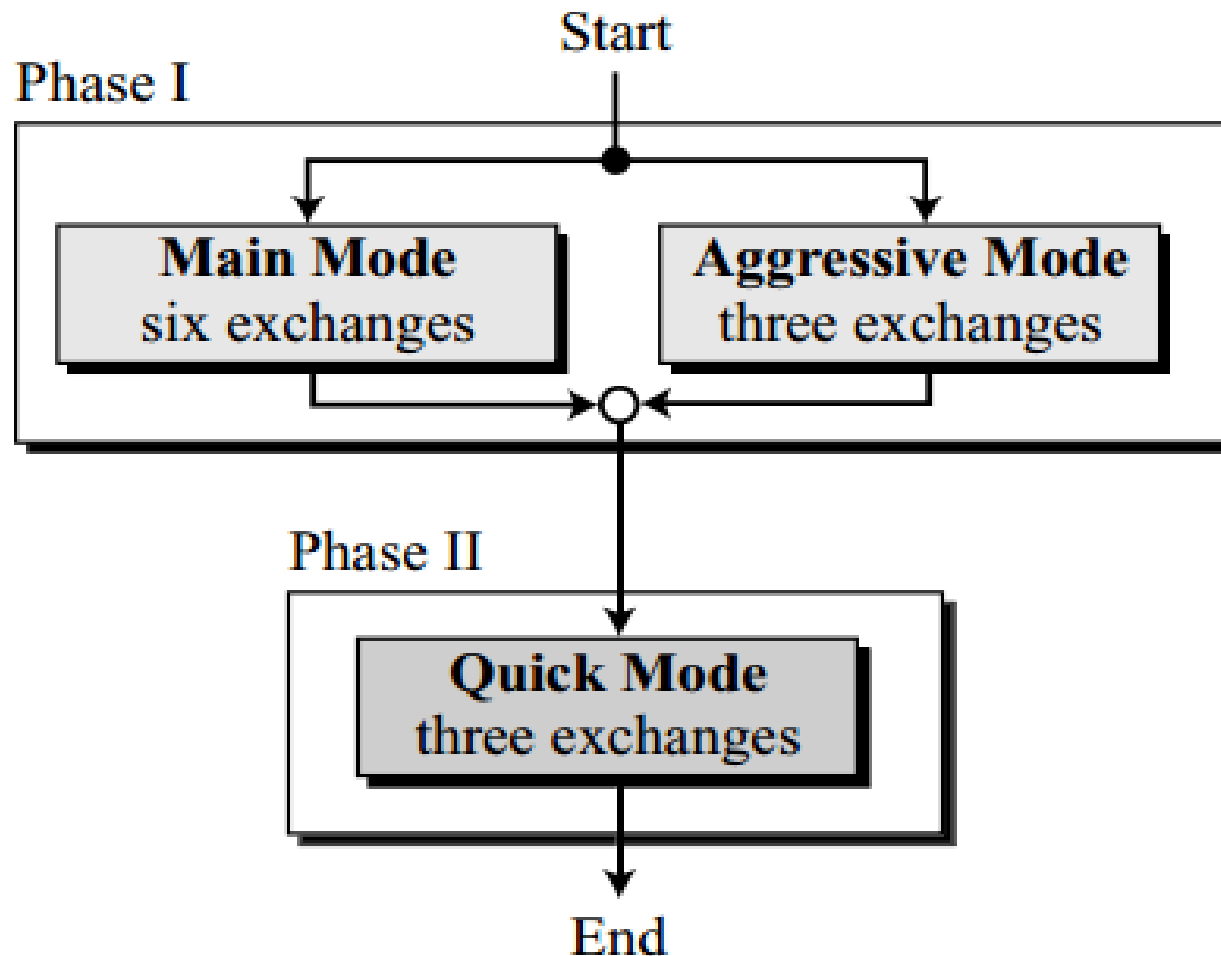
Improved Diffie-Hellman Key Exchange (with cookies)

- The cookie is the result of hashing a unique identifier of the peer (such as IP address, port number, and protocol), a secret random number known to the party that generates the cookie, and a timestamp.

Improved Diffie-Hellman Key Exchange (with cookies)



IKE Phases



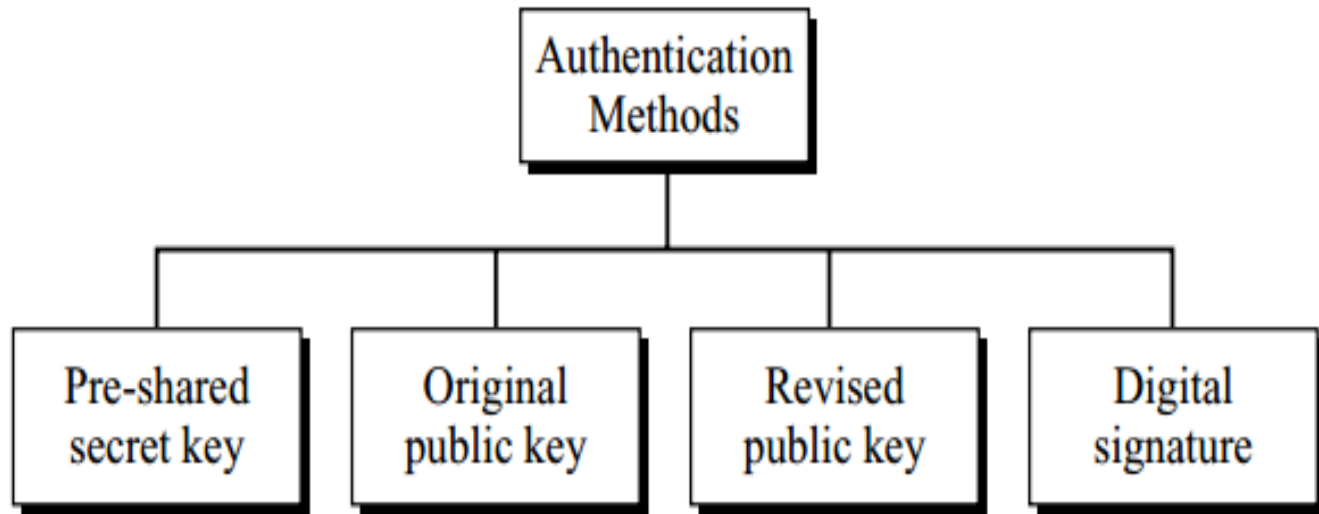
How It Works

- IKE operates in two phases
 - **Phase 1:** negotiate and establish an auxiliary end-to-end secure channel
 - Used by subsequent phase 2 negotiations
 - Only established once between two end points
 - **Phase 2:** negotiate and establish custom secure channels
 - Occurs multiple times
 - Both phases use Diffie-Hellman key exchange to establish a shared key

Main-mode or aggressive-mode methods

- Four different way to authenticate (either mode)
 - Digital signature
 - Two forms of authentication with public key encryption
 - Pre-shared key
- IKE does use public-key based cryptography for encryption

Main-mode or aggressive-mode methods



IKE Phase 1

- **Goal:** to establish a secure channel between two end points
 - This channel provides basic security features:
 - Source authentication
 - Data integrity and data confidentiality
 - Protection against replay attacks

IKE Phase 1

- **Rationale:** each application has different security requirements
- But they all need to negotiate policies and exchange keys!
- So, provide the basic security features and allow application to establish custom sessions

Example

- All packets sent to address [mybank.com](#) must be encrypted using 3DES with HMAC-MD5 integrity check
- All packets sent to address [www.forum.com](#) must use integrity check with HMAC-SHA1 ([no encryption is required](#))

Phase I: Main Mode

- The initiator and the responder exchange six messages.
 - In the first two messages, they exchange cookies (to protect against a clogging attack) and negotiate the SA parameters.
 - In the third and fourth messages, the initiator and responder usually exchange their half-keys and nonces

Phase I: Main Mode

- After exchanging the third and fourth messages, each party can calculate the common secret between them in addition to its individual hash digest.

$\text{SKEYID} = \textit{prf}(\text{preshared-key}, \text{N-I} \mid \text{N-R})$ (preshared-key method)

$\text{SKEYID} = \textit{prf}(\text{N-I} \mid \text{N-R}, g^{ir})$ (public-key method)

$\text{SKEYID} = \textit{prf}(\text{hash}(\text{N-I} \mid \text{N-R}), \text{Cookie-I} \mid \text{Cookie-R})$ (digital signature)

Other common secrets are calculated as follows:

$\text{SKEYID_d} = \textit{prf}(\text{SKEYID}, g^{ir} \mid \text{Cookie-I} \mid \text{Cookie-R} \mid 0)$

$\text{SKEYID_a} = \textit{prf}(\text{SKEYID}, \text{SKEYID_d} \mid g^{ir} \mid \text{Cookie-I} \mid \text{Cookie-R} \mid 1)$

$\text{SKEYID_e} = \textit{prf}(\text{SKEYID}, \text{SKEYID_a} \mid g^{ir} \mid \text{Cookie-I} \mid \text{Cookie-R} \mid 2)$

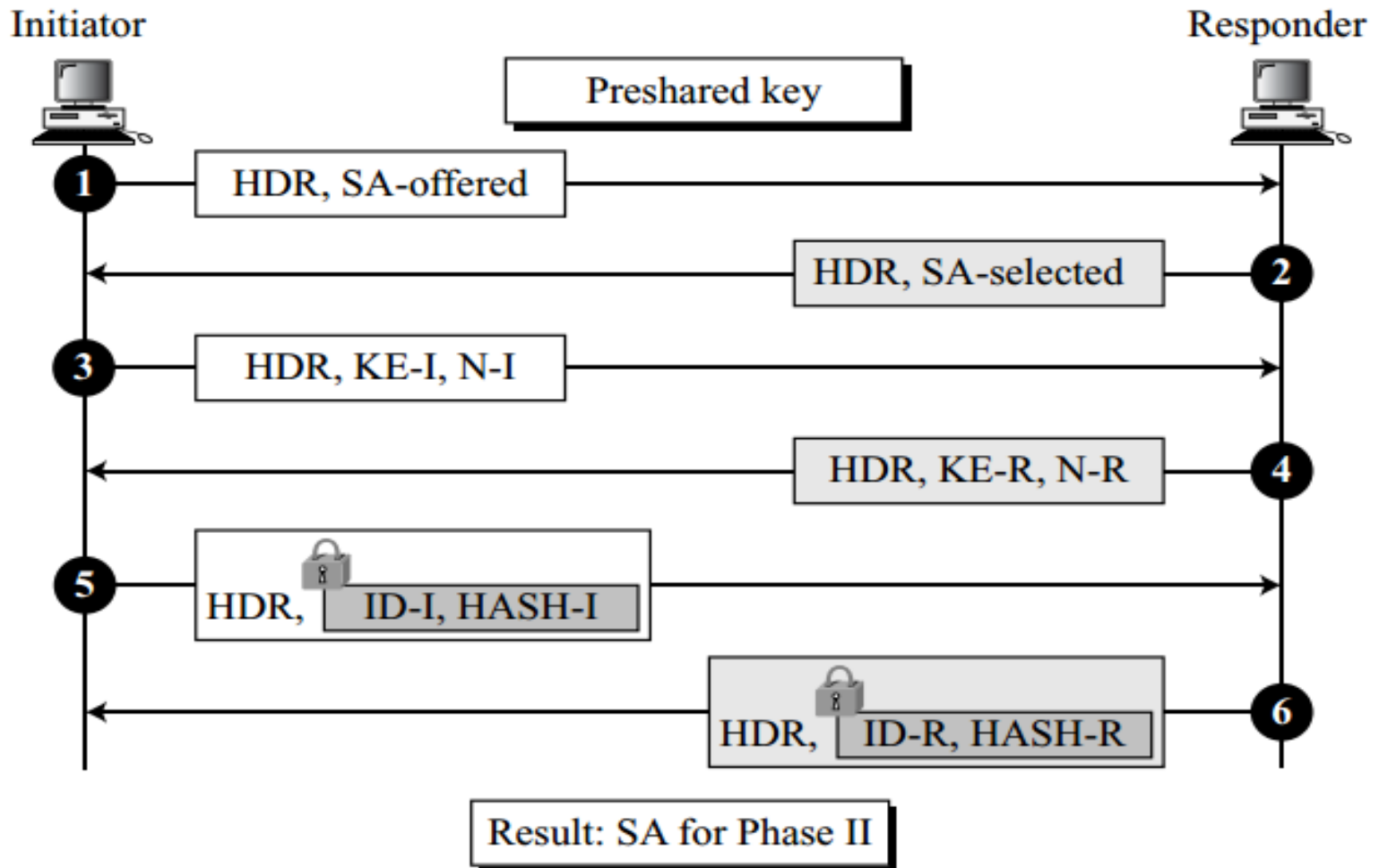
- The two parties also calculate two hash digests, HASH-I and HASH-R

$$\text{HASH-I} = \textit{prf}(\text{SKEYID}, \text{KE-I} \mid \text{KE-R} \mid \text{Cookie-I} \mid \text{Cookie-R} \mid \text{SA-I} \mid \text{ID-I})$$

$$\text{HASH-R} = \textit{prf}(\text{SKEYID}, \text{KE-I} \mid \text{KE-R} \mid \text{Cookie-I} \mid \text{Cookie-R} \mid \text{SA-I} \mid \text{ID-R})$$

Each party sends the hash to the other party to authenticate itself.

Main mode, preshared secret-key method



Main mode, preshared secret-key method

- The pre-shared key is the secret between Alice (initiator) and Bob (responder).
- Eve (intruder) does not have access to this key.
 - Eve cannot create SKEYID and therefore cannot create either HASH-I or HASH-R.
- IDs need to be exchanged in messages 5 and 6 to allow the calculation of the hash.

Main mode, original public-key method

- The initiator and the responder prove their identities by showing that they possess a private key related to their announced public key.

HDR: General header including cookies

KE-I (KE-R): Initiator's (responder's) half-key

N-I (N-R): Initiator's (responder's) nonce

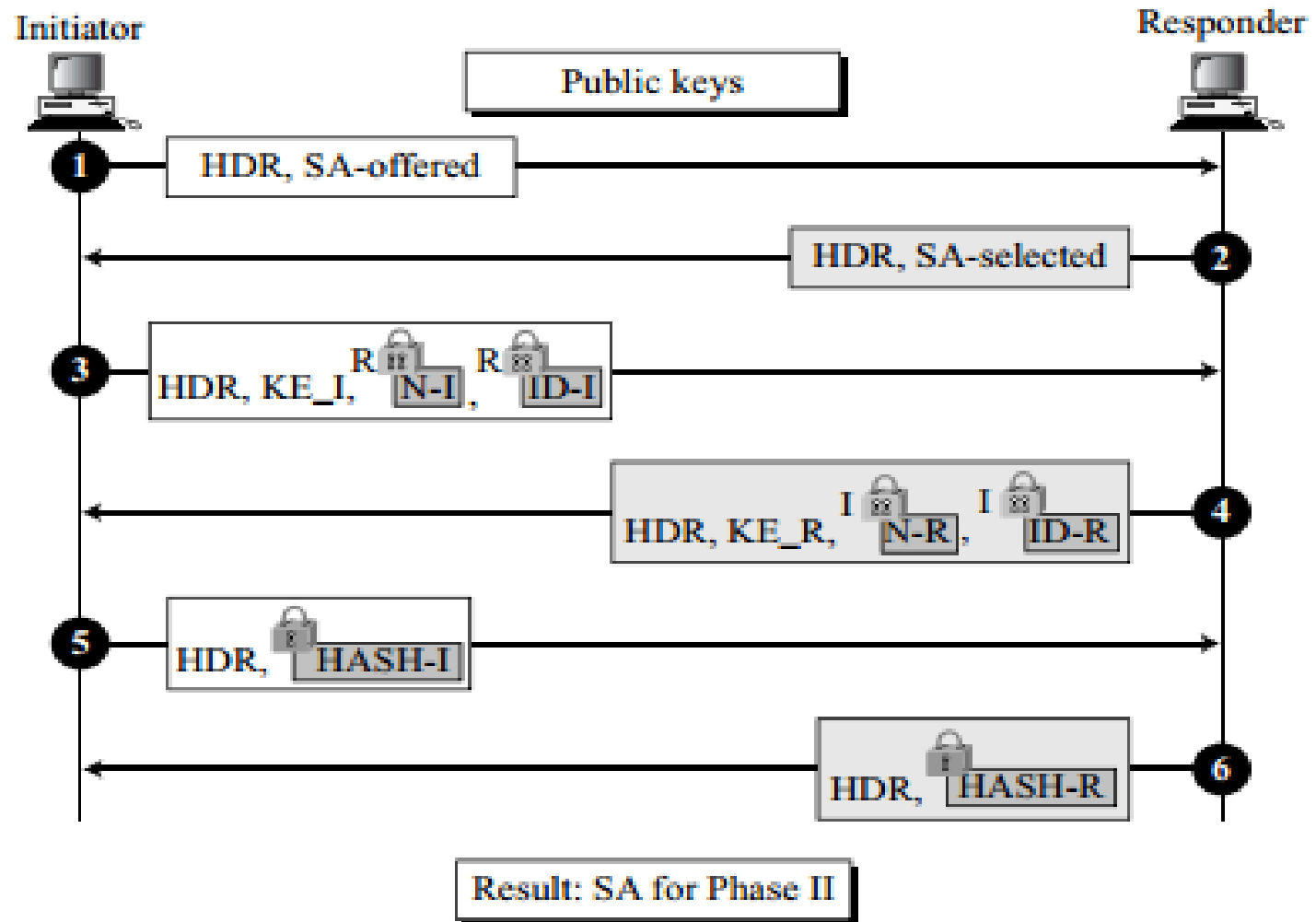
ID-I (ID-R): Initiator's (responder's) ID

HASH-I (HASH-R): Initiator's (responder's) hash

I  Encrypted with initiator's public key

R  Encrypted with responder's public key

 Encrypted with SKEYID_e



Main mode, original public-key method

- SKEYID is based on a hash of the nonces and the symmetric key.
- The hash of the nonces is used as the key for the keyed-HMAC function.
- SKEYID depends on the nonces and the nonces can only be decrypted by the private key (secret) of the receiver.
 - if the calculated hashes match those received, proof of each party is confirmed.

Main mode, revised public-key method

- The public key is used only to create a set of temporary secret keys.
- The two temporary secret keys, K-I and K-R, are calculated as

$$K-I = \textit{prf}(N-I, \text{Cookie-I})$$

$$K-R = \textit{prf}(N-R, \text{Cookie-R})$$

HDR: General header including cookies

KE-I (KE-R): Initiator's (responder's) half-key

Cert-I (Cert-R): Initiator's (responder's) certificate

N-I (N-R): Initiator's (responder's) nonce

ID-I (ID-R): Initiator's (responder's) ID

HASH-I (HASH-R): Initiator's (responder's) hash

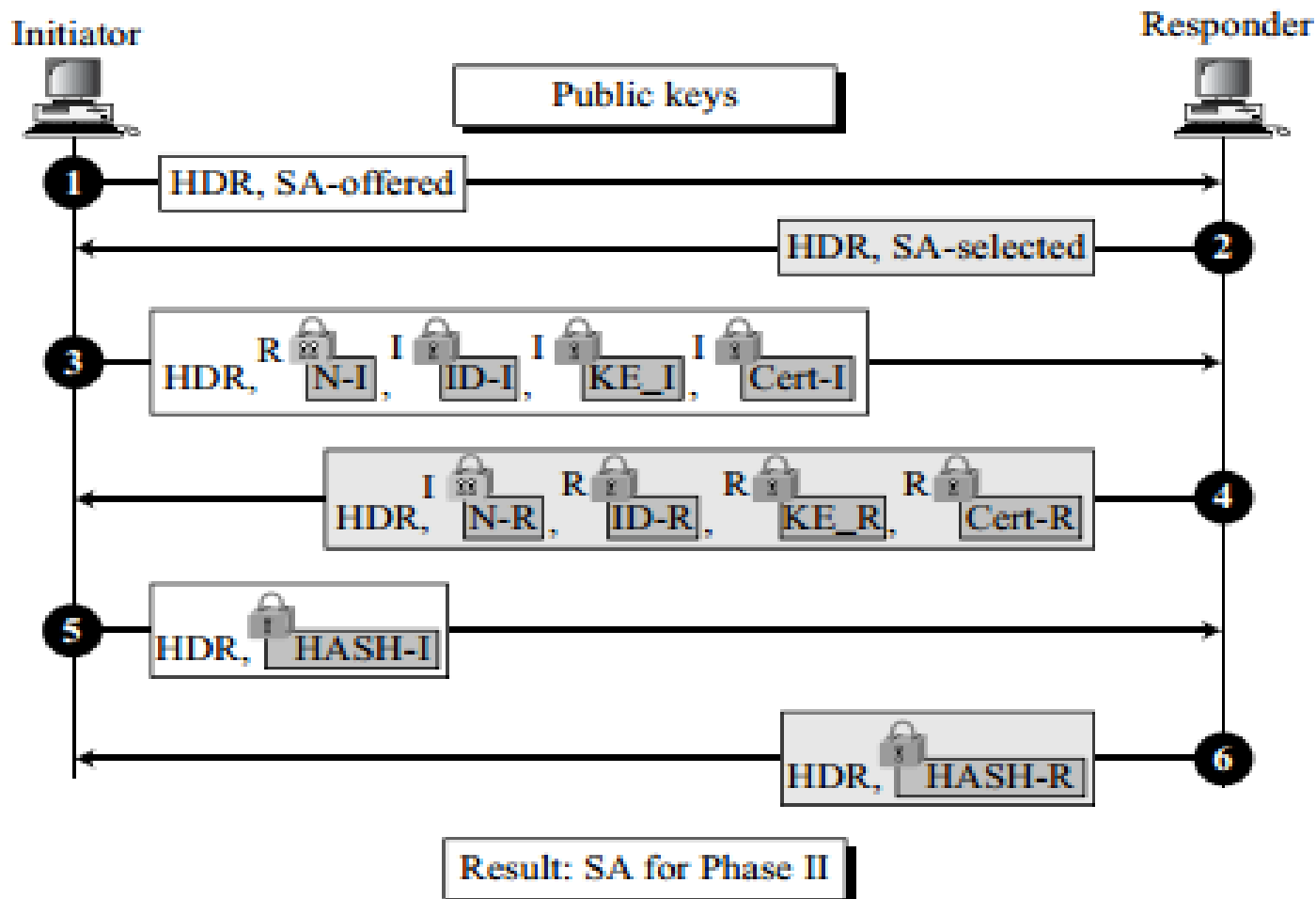
I  Encrypted with initiator's public key

R  Encrypted with responder's public key

R  Encrypted with responder's secret key

I  Encrypted with initiator's secret key

 Encrypted with SKEYID_e



Main mode, digital signature method

- Each party shows that it possesses the certified private key related to a digital signature.
- The sending of the certificates is optional.
 - The certificate can be sent here because it can be encrypted with SKEYID_e, which does not depend on the signature key.

HDR: General header including cookies

Sig-I: Initiator's signature on messages 1-4

Sig-R: Initiator's signature on messages 1-5

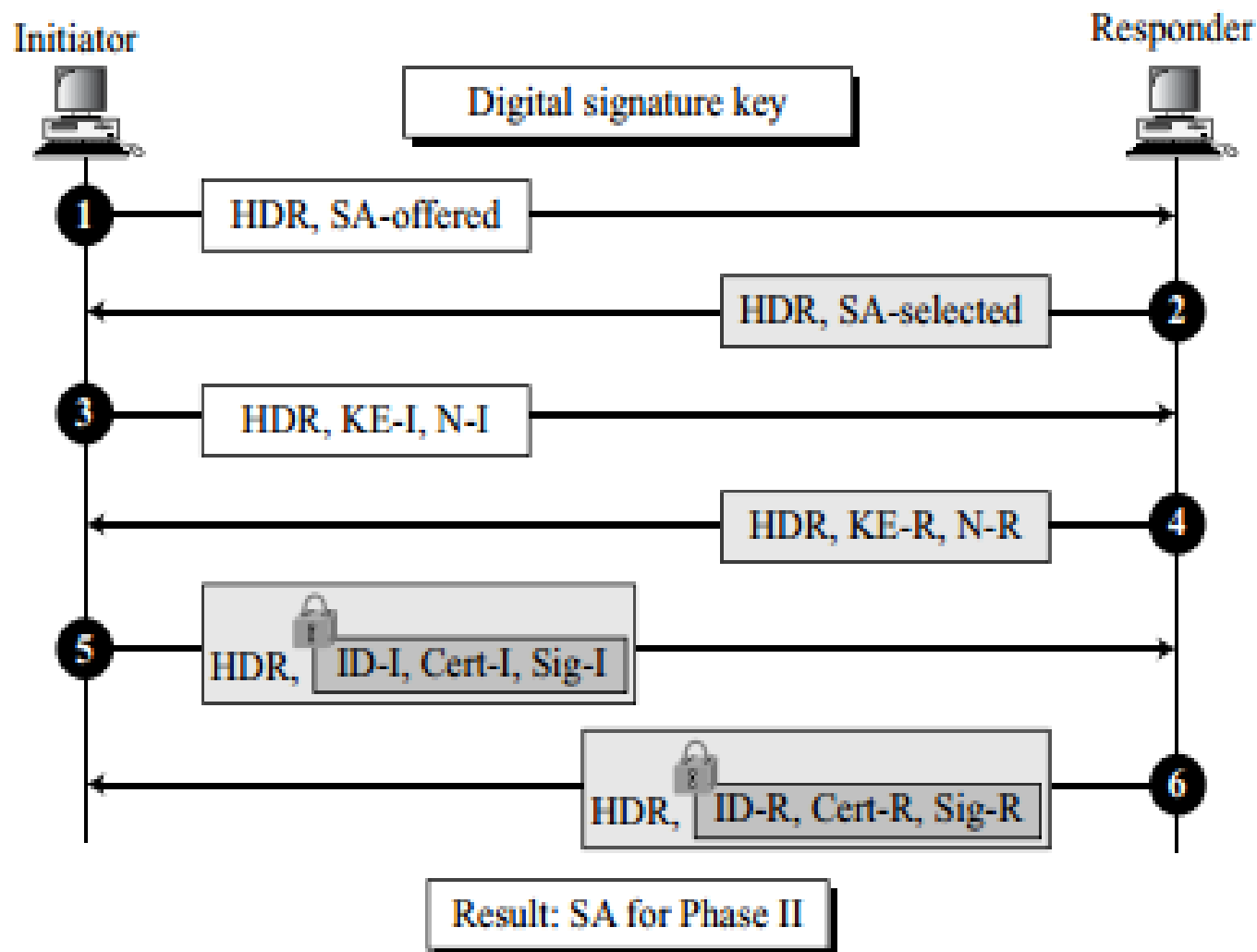
Cert-I (Cert-R): Initiator's (responder's) certificate

N-I (N-R): Initiator's (responder's) nonce

KE-I (KE-R): Initiator's (responder's) half-key

ID-I (ID-R): Initiator's (responder's) ID

 Encrypted with SKEYID_e



Phase I: Aggressive Mode

- Only three messages are exchanged.
- Each aggressive mode is a compressed version of the corresponding main mode.
 - Messages 1 and 3 are combined to make the first message.
 - Messages 2, 4, and 6 are combined to make the second message.
 - Message 5 is sent as the third message.

Aggressive mode, preshared-key method

KE-I (IK-R): Initiator's (responder's) half-key

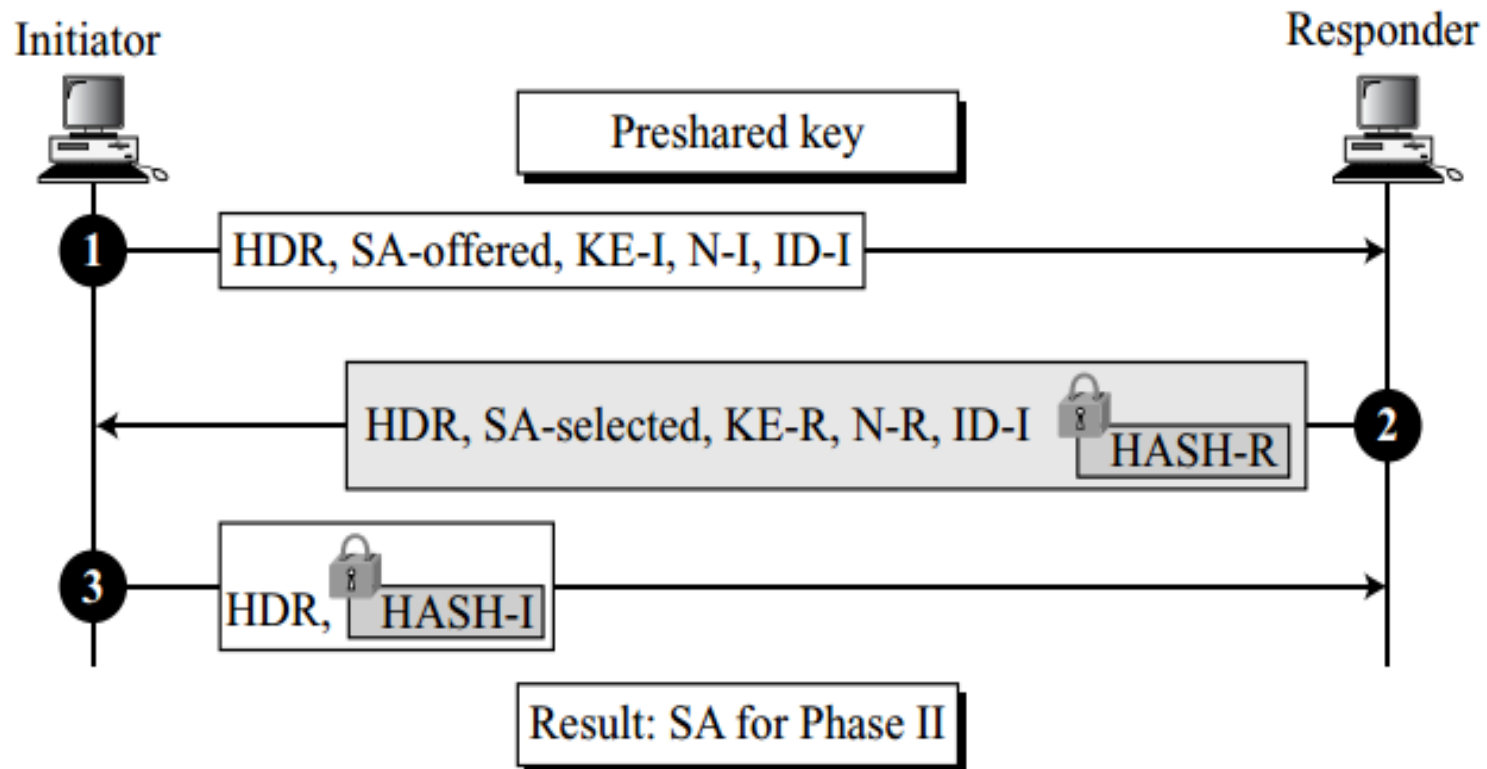
N-I (N-R): Initiator's (responder's) nonce

HASH-I (HASH-R): Initiator's (responder's) hash

HDR: General header including cookies

 Encrypted with SKEYID_e

ID-I (ID-R): Initiator's (responder's) ID



Aggressive mode, preshared-key method

- After receiving the first message, the responder can calculate SKEYID and HASH-R.
- The initiator cannot calculate SKEYID until it receives the second message.
- HASH-I in the third message can be encrypted.

Aggressive mode, original public-key method

- the responder can calculate the SKEYID and HASH-R after receiving the first message
 - but the initiator must wait until it receives the second message.

HDR: General header including cookies

KE-I (KE-R): Initiator's (responder's) half-key

N-I (N-R): Initiator's (responder's) nonce

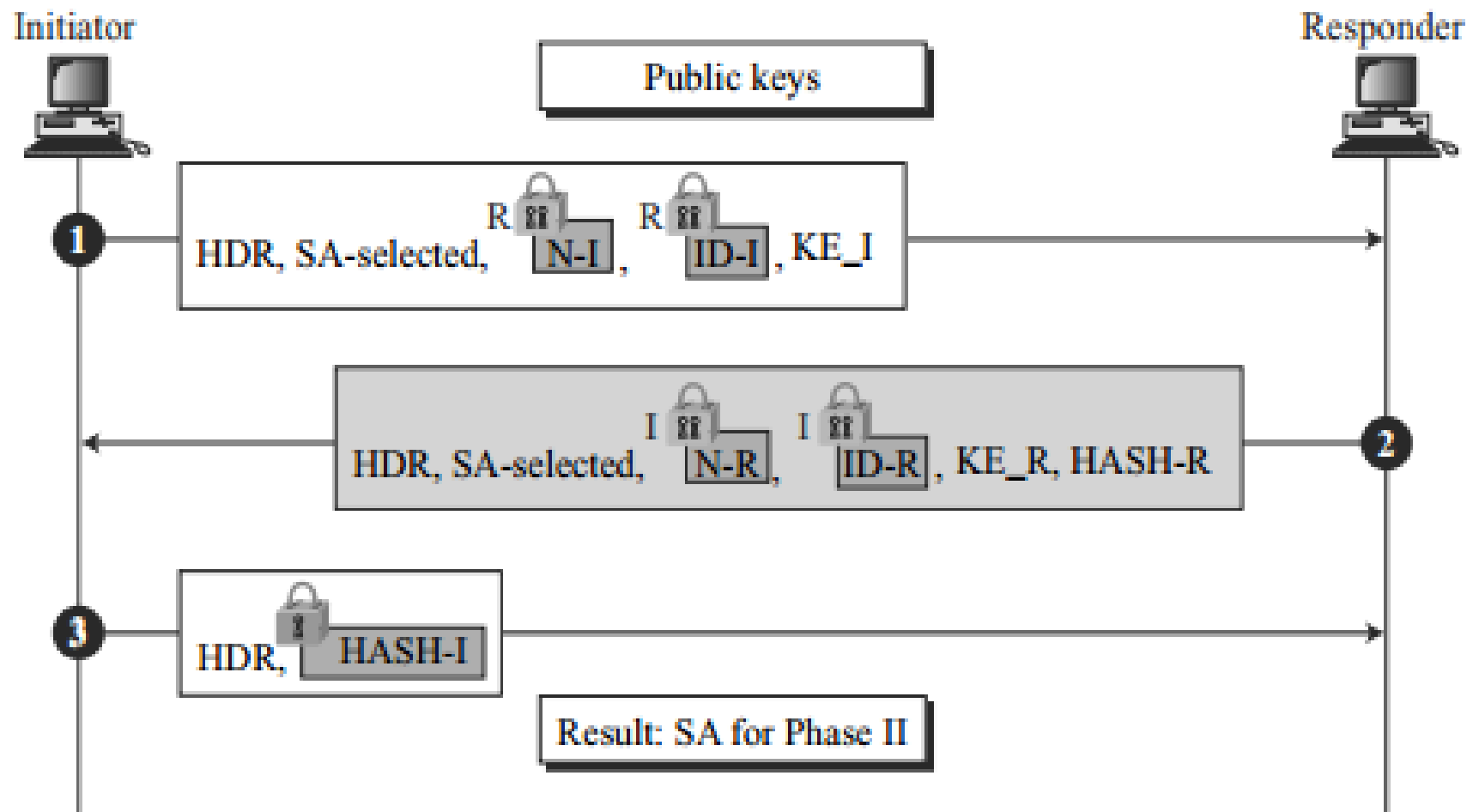
ID-I (ID-R): Initiator's (responder's) ID

I  Encrypted with initiator's public key

R  Encrypted with responder's public key

 Encrypted with SKEYID_e

HASH-I (HASH-R): Initiator's (responder's) hash

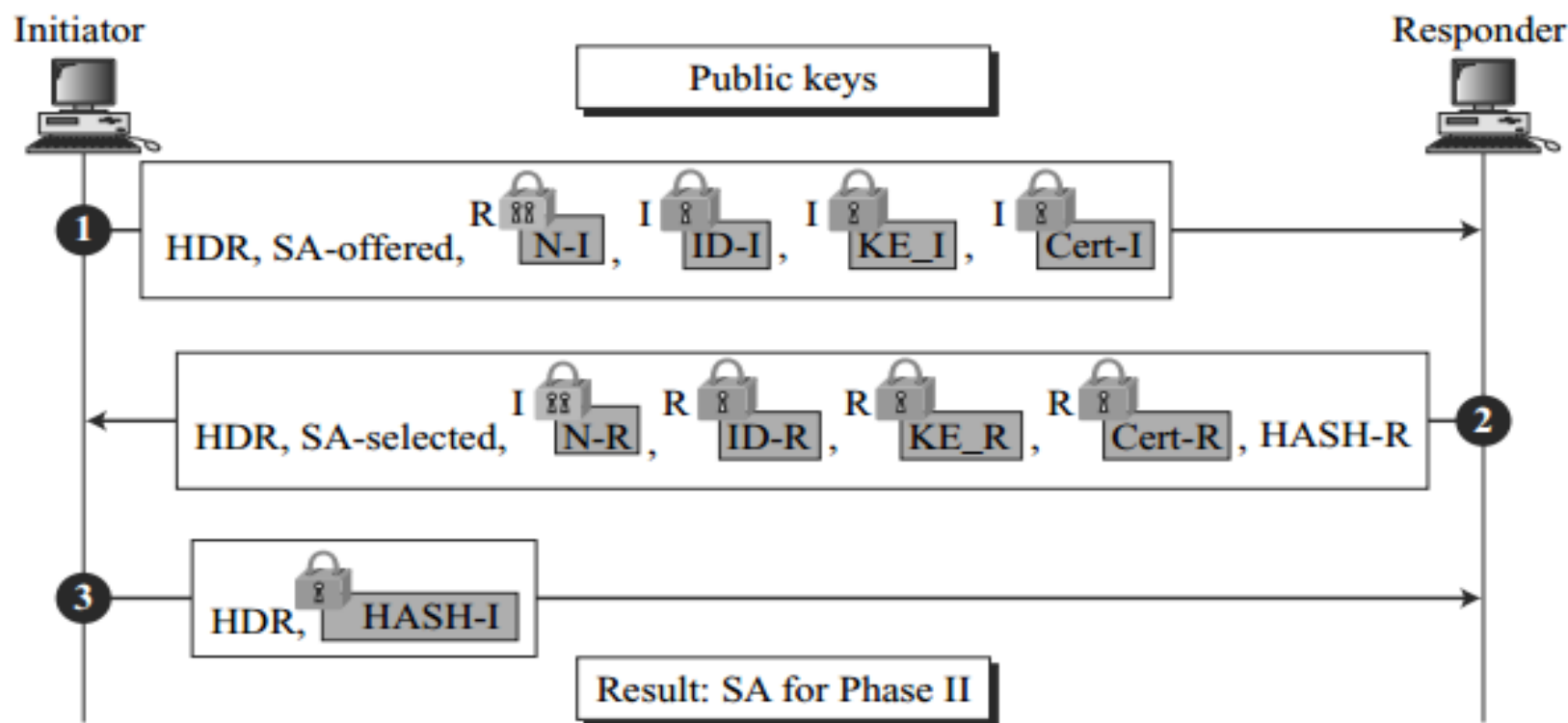


Aggressive mode, revised public-key method

- The idea is the same as for the main mode
 - except that some messages are combined.

HDR: General header including cookies
 KE-I (KE-R): Initiator's (responder's) half-key
 Cert-I (Cert-R): Initiator's (responder's) certificate
 N-I (N-R): Initiator's (responder's) nonce
 ID-I (ID-R): Initiator's (responder's) ID
 HASH-I (HASH-R): Initiator's (responder's) hash

I  Encrypted with initiator's public key
 R  Encrypted with responder's public key
 R  Encrypted with responder's secret key
 I  Encrypted with initiator's secret key
 Encrypted with SKEYID_e



Agressive mode, digital signature method



Encrypted with SKEYID_e

Sig-I (Sig-R): Initiator's (responder's) signature

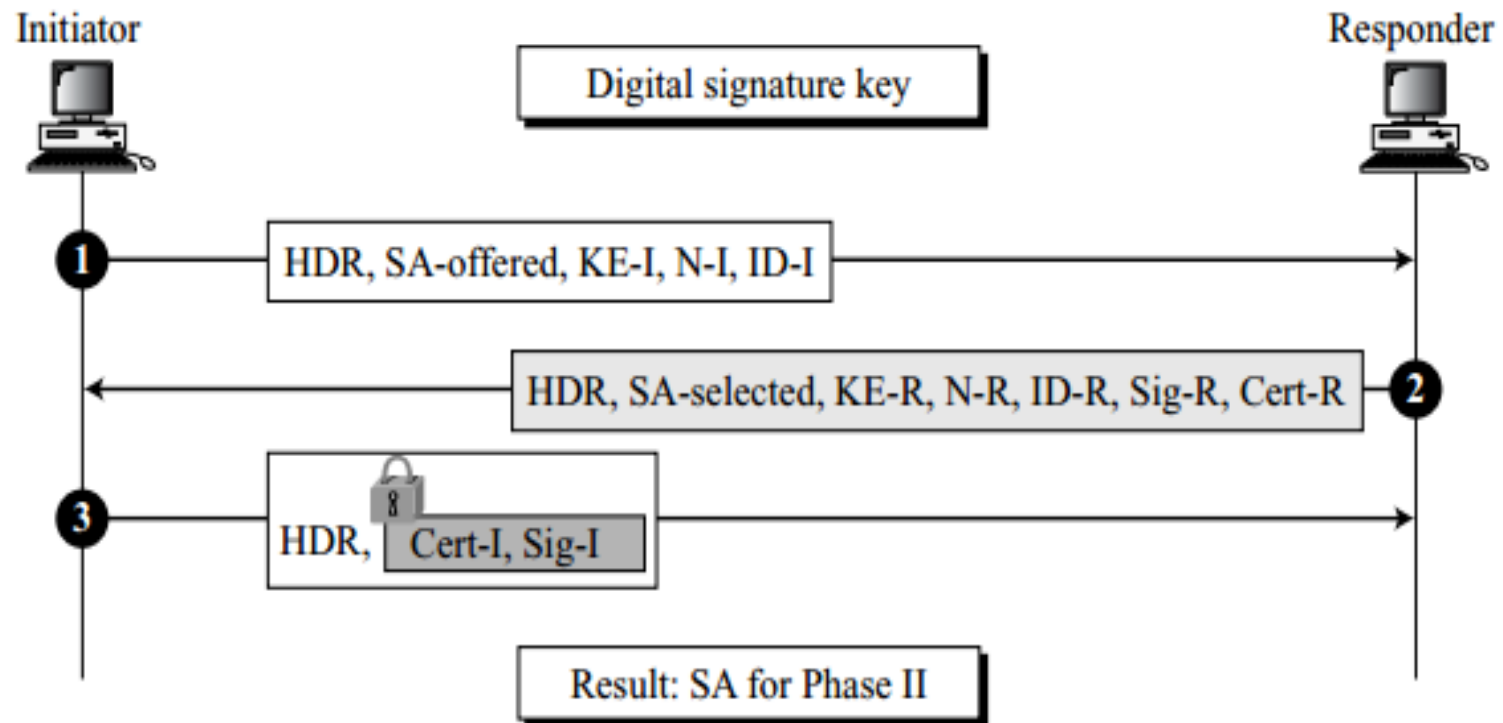
N-I (N-R): Initiator's (responder's) nonce

HDR: General header including cookies

KE-I (KE-R): Initiator's (responder's) half-key

Cert-I (Cert-R): Initiator's (responder's) certificate

ID-I (ID-R): Initiator's (responder's) ID



Phase II: Quick Mode

- After SAs have been created in either the main mode or the aggressive mode, phase II can be started.
- The quick mode uses IKE SAs to create IPSec SAs (or SAs for any other protocol).
- **Either party can be the initiator**
 - initiator of phase II can be the initiator of phase I or the responder of phase I.

Phase II: Quick Mode

KE-I (KE-R): Initiator's (responder's) half-key

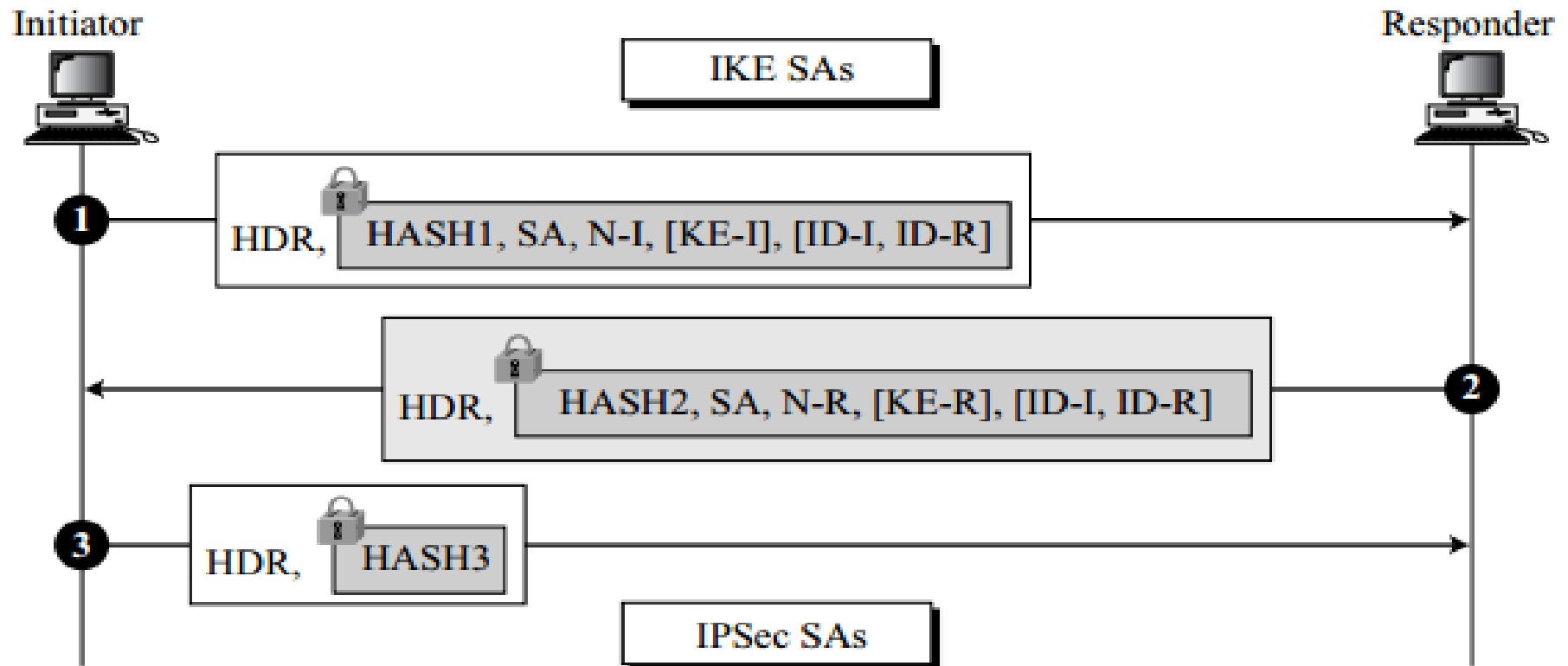
N-I (N-R): Initiator's (responder's) nonce

ID-I (ID-R): Initiator's (responder's) ID

HDR: General header including cookies

 Encrypted with SKEYID_e

SA: Security association



Phase II: Quick Mode

The messages are authenticated using three keyed-HMACs: HASH1, HASH2, and HASH3.

$$\text{HASH1} = \textit{prf}(\text{SKEYID_d}, \text{MsgID} \mid \text{SA} \mid \text{N-I})$$

$$\text{HASH2} = \textit{prf}(\text{SKEYID_d}, \text{MsgID} \mid \text{SA} \mid \text{N-R})$$

$$\text{HASH3} = \textit{prf}(\text{SKEYID_d}, 0 \mid \text{MsgID} \mid \text{SA} \mid \text{N-I} \mid \text{N-R})$$

Perfect Forward Security (PFS)

- KE allows Perfect Forward Security (PFS) as an option.
 - an additional Diffie-Hellman half-key is exchanged and the resulting shared key is used in the calculation of **key material**
 - PFS is effective if the Diffie-Hellman key is immediately deleted after the calculation of the key material for each quick mode.

- Key Materials

After the exchanges in phase II, an SA for IPSec is created including the key material, K , that can be used in IPSec.

$$\begin{array}{ll} K = \textit{prf}(\text{SKEYID_d}, \text{protocol} \mid \text{SPI} \mid \text{N-I} \mid \text{N-R}) & \text{(without PFS)} \\ K = \textit{prf}(\text{SKEYID_d}, g^{ir} \mid \text{protocol} \mid \text{SPI} \mid \text{N-I} \mid \text{N-R}) & \text{(with PFS)} \end{array}$$

- If the length of K is too short for the particular cipher selected, then
 - a sequence of keys is created
 - each key is derived from the previous one
 - the keys are concatenated to make a longer key

$$\begin{aligned} K_1 &= \textit{prf}(\text{SKEYID_d}, \text{protocol} \mid \text{SPI} \mid \text{N-I} \mid \text{N-R}) \\ K_2 &= \textit{prf}(\text{SKEYID_d}, K_1 \mid \text{protocol} \mid \text{SPI} \mid \text{N-I} \mid \text{N-R}) \\ K_3 &= \textit{prf}(\text{SKEYID_d}, K_2 \mid \text{protocol} \mid \text{SPI} \mid \text{N-I} \mid \text{N-R}) \\ &\dots \\ K &= K_1 \mid K_2 \mid K_3 \mid \dots \end{aligned}$$

- The key material created is **unidirectional**
 - each party creates different key material because the SPI used in each direction is different.